Onboarding Problem

2D Heat Diffusion on a Uniform Grid

University of Waterloo High-Performance Computing Team

Overview

You will implement two components of a 2D heat diffusion solver:

1. A **grid class** that stores the 2D temperature field in memory.
2. The **stencil kernel** that advances the simulation by one time step.

Everything else, i.e., the time integrator, boundary conditions, initial conditions, and I/O, is provided. You DO NOT need to know any thermal physics or theory.

The Math

The Governing Heat Equation

Heat spreads across a 2D surface according to the following Partial Differential Equation:

$$\frac{\partial T}{\partial t} \;=\; \alpha\left(\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2}\right) \tag{1}$$

where $T(x, y, t)$ is the temperature and α is a constant. Again, you do not need to understand the physics, math, theory, or origin of the equation; this is merely for context.

Discretization onto a Grid

Replace the continuous derivatives with finite differences:

$$T_{t+1}(i,j) = T_t(i,j) + \Delta t \cdot \alpha\left(\frac{T_t(i{+}1,j) - 2T_t(i,j) + T_t(i{-}1,j)}{\Delta x^2} + \frac{T_t(i,j{+}1) - 2T_t(i,j) + T_t(i,j{-}1)}{\Delta y^2}\right) \tag{2}$$

Given Constants

With constants α, Δx, Δy, and Δt chosen for stability, the equation simplifies to the following:

$$\boxed{T_{t+1}(i,j) \;=\; 0.5 \cdot T_t(i,j) \;+\; 0.125 \cdot \big(T_t(i{-}1,j) + T_t(i{+}1,j) + T_t(i,j{-}1) + T_t(i,j{+}1)\big)} \tag{3}$$

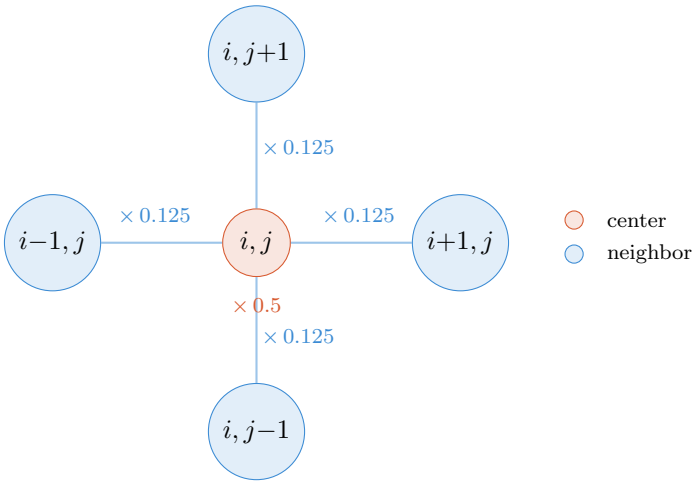
Converting Equations to Code

Notice how this is just a weighted average. In code, the above could look something like this:

```
new_arr[i][j] = 0.5   *  old_arr[i][j] +
                0.125 * (old_arr[i-1][j] +
                         old_arr[i+1][j] +
                         old_arr[i][j-1] +
                         old_arr[i][j+1]
                        );
```

The Five-Point Stencil

To provide a more visual understanding and intuition of what is happening, consider the diagram below. You can picture the heat from each "bubble" (grid cell) leaking into its neighbors.



Task 1: The Grid Class

Design and implement a `Grid` class that stores a 2D field of `double` values of size $N_x \times N_y$. While you may expand as much as you wish, your class must satisfy the following interface:

```cpp
#include <cstddef>

class Grid {
private:
    std::size_t Nx_;
    std::size_t Ny_;

public:
    Grid(std::size_t nx, std::size_t ny);

    std::size_t Nx() const;
    std::size_t Ny() const;

    // Element access; implement both overloads.
    double& operator()(std::size_t i, std::size_t j);       // read/write
    double  operator()(std::size_t i, std::size_t j) const; // read
};
```

The two `operator()` overloads give read/write and read-only access to the cell at row `i`, column `j`. The evaluation harness uses only these overloads to set initial conditions and read back your results, so they must work no matter how you store the field internally. The implementation of everything else is up to you. We want to see how you think and how you approach such problems.

Requirements:

1. Default-initialized to zero.
2. Implement both `operator()` overloads; the evaluation harness relies on them.

3. You are free to choose the memory layout and overall design of the Grid class. Be prepared to discuss your decisions during the interview.

Task 2: The Stencil Kernel

Implement the function that applies the five-point stencil over all interior grid points ($1 \leq i < N_x - 1,\ 1 \leq j < N_y - 1$).

Boundary values must remain unchanged. You are not required to implement any boundary conditions; simply copy the boundary values from the old grid to the new grid.
The boundary copy below is illustrative; the interior update is up to you.

```
void apply_stencil(const Grid& old_grid, Grid& new_grid) {
    // Copy boundaries
    for (std::size_t i{}; i < old_grid.Nx(); ++i) {
        new_grid(i, 0) = old_grid(i, 0);
        new_grid(i, old_grid.Ny() - 1) = old_grid(i, old_grid.Ny() - 1);
    }

    for (std::size_t j{}; j < old_grid.Ny(); ++j) {
        new_grid(0, j) = old_grid(0, j);
        new_grid(old_grid.Nx() - 1, j) = old_grid(old_grid.Nx() - 1, j);
    }

    // Your implementation...
}
```

Deliverables

All testing and benchmarking will be performed automatically on team infrastructure. Submit a link to a fork of the provided repository containing your implementation of:

1. `grid.hpp` — containing your Grid implementation.
2. `stencil.cpp` — containing your stencil implementation.

Evaluation

Submissions will be evaluated on correctness, implementation quality, design decisions, and performance. Performance will be measured using wall clock execution time on a team benchmark machine.

Correctness alone is not sufficient. We expect thoughtful consideration of memory layout, performance, and overall design. Applications are reviewed holistically, and advancement is not determined solely by benchmark results.

You may use any C++17 or later standard library facilities and introduce additional methods, helper functions, or internal data structures as needed.

Submission

After your submission, we will invite you to a short virtual chat to discuss your design further. This is solely to get an idea of your thinking and how you approach problems.

On Artificial Intelligence

We encourage using AI to learn and explore ideas. That said, your submission should be your own work; copy-pasting from AI will be obvious, especially during the chat.

Good luck.